

AISSD5000 KV Cache 跨实例热共享验证报告 —— fs:// 共享池 · 480B · TP4×2

测试平台：AMD Instinct MI308X ×8（每卡 192 GB HBM） / ROCm 7.2 / vLLM 0.20.1+rocm721 + LMCache v1（上游主线源码编译）

被测存储：AISSD5000（WS5000）全闪 NVMe-oF 阵列（4 盘 RAID0，RoCEv2 over 100GbE，XFS，14 TB）

模型：Qwen3-Coder-480B-FP8（MoE，权重约 450 GB）

部署形态：双实例 TP4×2（实例 A：卡 0-3 / 端口 8000；实例 B：卡 4-7 / 端口 8001）

负载：长上下文冷恢复——每会话约 29.8K token，KV ≈ 7.15 GB/会话

日期：2026-07-06

关联报告：《AISSD5000-KVCache性能测试报告（480B · TP4x2双实例 · 正式版）》§ 4.6 遗留问题的闭环验证

一、背景与目的

TP4×2 双实例测试（2026-07-06 上午）发现：LMCache `LocalDiskBackend` 的键索引在进程内存中维护，实例 B 无法命中实例 A 灌入到共享阵列的 KV（hit=0，退化重算，TTFT 67 s）——跨实例热共享存在软件层缺口。

方案分析确定了修复路径：改用 LMCache 内置的 `fs:// remote connector (FSConnector)`。其索引即文件系统本身（`contains()` 直接 stat 文件），无进程内状态，理论上天然支持跨实例、跨重启共享。本轮目的：

- 可行性核实：确认该版本代码中 fs connector 的配置接线、注册、元数据完整性；
- 功能验证：双实例挂同一共享池，交叉冷读对方灌入的会话，验证全量命中；
- 性能验证：与 `LocalDiskBackend`（上午同机同负载基线）同口径对比，确认共享不以性能为代价。

结论摘要：方案完全成立。

- 跨实例热共享打通：A、B 双实例同时交叉冷读对方灌入的 32 个会话（共 229 GB），32/32 全量磁盘命中、零未命中，TTFT p50 27.1-27.4 s，与读自己灌入的会话（25.0-26.7 s）基本无差——共享代价约 5%，接近零；
- 性能与 `LocalDiskBackend` 完全打平：同口径两档（全机并发 16/32）TTFT、聚合吞吐、阵列供给全部一致（10.4 GB/s 峰值、9.0-9.2 GB/s 忙窗均值），共享能力不以性能为代价；
- 一份 KV、全机复用成为现实：任何实例预填充过的长上下文，其余实例即时可用——会话可在实例间自由迁移/负载均衡，预填充成本全机只付一次；
- 发现并绕过一个上游 bug：文档示例 `fs:///path` 过不了该版本的 URL 校验（强制要求 `host:port`），需写成 `fs://local:0/path` 占位形式（本报告附复现细节，可向上游提 issue）。

二、可行性核实（代码级）

对同事服务器容器内 LMCache 源码（`/root/LMCache/lmcache`，上游主线 2026-06-29）的核实结果：

核 实 项	位置	结论
配置 接线	<code>v1/config.py</code> L114 - 119	<code>remote_url/remote_serde</code> 支持环境变量 <code>LMCACHE_REMOTE_URL / LMCACHE_REMOTE_SERDE</code> （标记 deprecated 但功能完整）
连接 器注 册	<code>connector/fs_adapter.py</code>	<code>FsConnectorAdapter</code> 匹配 <code>fs://</code> 前缀，从 URL 提取目录路径
索引 机制	<code>connector/fs_connector.py</code> <code>exists()</code>	直接 <code>os.path.exists(file)</code> ——索引即文件系统，无进程内状态
尾块 元数 据	<code>connector/base_connector.py</code> L55 - 58	<code>save_chunk_meta</code> 默认 True：每文件带 28 字节头 (shape/dtype/fmt)，不满 256 token 的尾块可无损恢复
写入 原子 性	<code>fs_connector.py</code> <code>put()</code>	临时文件写入 + <code>os.replace()</code> 原子改名——读方不会读到半个文件
落地 确认	实例日志	4 worker 均 <code>Creating FS connector + Connection</code> <code>initialized at fs://local:0/mnt/ws5000/kvpool_fs</code>

发现的上游 bug：`parse_remote_url()`（`connector/__init__.py` L67 - 68）对所有 URL 强制 `assert host/port` 非空，而文档示例给的是无 host 的 `fs:///tmp/lmcache`——按文档配置会导致 RemoteBackend 每 30 s 重试建连并永久失败（KV 静默丢弃、池目录零文件，仅在 WARNING 日志可见）。容器内单测确认绕法：`fs://local:0/mnt/ws5000/kvpool_fs`（占位 `host:port`，fs adapter 只取 path）解析成功。

三、实验设计

3.1 配置（与上午 TP4×2 基线仅存储后端不同）

```
LMCACHE_CHUNK_SIZE=256 LMCACHE_LOCAL_CPU=True LMCACHE_MAX_LOCAL_CPU_SIZE=4
LMCACHE_REMOTE_URL=fs://local:0/mnt/ws5000/kvpool_fs # 两实例同一共享池
LMCACHE_REMOTE_SERDE=naive
（不再配置 LMCACHE_LOCAL_DISK）
```

其余与基线完全一致：TP4×2 双实例、`--no-enable-prefix-caching`、`gpu-memory-utilization 0.9`、`max-model-len 32768`、`PYTHONHASHSEED=0`。

3.2 数据准备与三档测量

- 双实例并行灌入 48 个互异会话（A 灌 0 - 23、B 灌 24 - 47），共享池实测落盘 344 GB / 22656 文件；
- 每档测量前 `sync; echo 3 > /proc/sys/vm/drop_caches` 清 1.5 TB 页缓存，保证物理冷读；

- 档1 OWN 8+8：各读自己灌的会话（A 读 0 - 7、B 读 24 - 31）——与基线同口径；
- 档2 OWN 16+16：同上拉满（A 读 0 - 15、B 读 24 - 39）；
- 档3 CROSS 16+16（热共享证明）：交叉冷读——A 读 B 灌的 24 - 39，B 读 A 灌的 0 - 15，两实例读取集合互不相交且都不是自己写的；
- 取证：逐请求统计 `hit tokens: 30xxx`（满额命中）与 `hit tokens: 0`（未命中），并行采集 `iostat -x 1 /dev/md0`。

四、测试结果

4.1 三档总表（全部物理冷读）

档位	读取关系	TTFT p50 A / B (s)	聚合吞吐	盘峰值	盘忙窗均值	满命中取证
OWN 8+8	各读自己灌的	13.73 / 13.59	67.4 tok/s	10.37 GB/s	9.21 GB/s	16/16，零未命中
OWN 16+16	各读自己灌的	26.70 / 25.03	71.6 tok/s	10.36 GB/s	8.98 GB/s	32/32，零未命中
CROSS 16+16	互读对方灌的	27.38 / 27.09	69.9 tok/s	10.41 GB/s	9.22 GB/s	32/32，零未命中

（聚合吞吐 = $2 \times N \times 64 \div$ 两实例较慢者整批耗时）

4.2 热共享验证（核心结果）

- 32 个交叉请求全部满额磁盘命中（每请求 `hit tokens = 30208`，命中率 100%），无一退化重算；
- 交叉读 vs 读自己：TTFT p50 27.2 s vs 25.9 s（均值口径），共享代价约 5%——来源仅是 `stat/` 调度的微小差异，数据路径完全相同；
- 交叉读时阵列供给不降反平：峰值 10.41 GB/s、忙窗 9.22 GB/s，为三档最高——共享池模式下阵列供给能力不受读取关系影响；
- 对照昨日 `LocalDiskBackend` 的跨实例尝试（`hit=0`、TTFT 67 s、退化重算）：同一问题，换后端即闭环。

4.3 性能同口径对照：fs:// 共享池 vs LocalDiskBackend（上午基线）

指标（全机并发 16 档）	LocalDiskBackend	fs:// 共享池
TTFT p50 (A/B)	13.17 / 13.89 s	13.73 / 13.59 s
聚合吞吐	66.5 tok/s	67.4 tok/s
盘峰值 / 忙窗均值	10.33 / 8.55 GB/s	10.37 / 9.21 GB/s

指标（全机并发 32 档）	LocalDiskBackend	fs:// 共享池
TTFT p50 (A/B)	26.01 / 27.80 s	26.70 / 25.03 s

聚合吞吐	69.0 tok/s	71.6 tok/s
盘峰值 / 忙窗均值	10.27 / 8.88 GB/s	10.36 / 8.98 GB/s

两后端在两档上 TTFT、吞吐、阵列供给全部在测量噪声内一致 (fs:// 甚至略优)。结论：获得跨实例共享能力，性能零代价。fs connector 的 asyncio+aiofiles 读路径与 LocalDiskBackend 线程池读路径在本负载下同样能把阵列拉到单口线速的 90%+。

4.4 灌入（写路径）

双实例并行灌入 48 会话耗时 363 s，与基线（361 - 363 s）一致；写路径为异步 put（临时文件+原子改名），22656 个文件全部完整落盘，无损坏、无残留 .tmp。

五、价值与边界

5.1 对生产架构的直接意义

- 1. 预填充成本全机只付一次：任何实例处理过的长上下文（长系统提示、RAG 文档、历史会话），其余实例即时命中——多实例集群的 KV 复用率从“实例内”提升为“全机”；
- 2. 会话可自由迁移：会话下一轮请求被路由到任何实例都能以 27 s（而非重算的 270 s）恢复 30K 上下文——负载均衡器无需会话亲和性约束；
- 3. 实例滚动重启不再清空缓存：池在阵列上、索引即文件系统，实例重启后所有历史 KV 仍然可见（LocalDiskBackend 重启即全丢）；
- 4. 配合 AISSD5000 的容量与带宽：14 TB 起步的共享池 + 单口 10.4 GB/s（可扩展至 6 口）供给，是本地盘方案（每机 2 TB、6.78 GB/s、无法跨实例语义共享）不具备的架构位。

5.2 使用边界（工程约束，均已在实验中控制）

- 1. 同机多实例即插即用；跨节点需共享文件系统层——AISSD5000 是块设备，多主机同时挂同一 XFS 会损坏文件系统；跨节点共享需 NFS 网关/集群文件系统承载，或改用集中式索引后端（Mooncake/Redis 类），列为后续验证；
- 2. 共享各方几何必须一致：键格式含 world_size/worker_id/chunk hash——同 TP 并行度、同 chunk size、同 PYTHONHASHSEED、同模型；
- 3. 池生命周期需外部管理：remote 池无 LRU 自动逐出（MAX_LOCAL_DISK_SIZE 不适用），需按目录/mtime 定期清理；14 TB 池按本负载可容纳约 1900 个 30K 会话，短期无压力；
- 4. 上游 URL 校验 bug：必须写 fs://local:0/path 占位形式（§2），建议向上游提 issue。

六、结论

- 1. 方案一可行且已验证：LMCache 内置 fs:// remote connector + AISSD5000 共享池，零代码改动打通跨实例 KV 热共享——交叉冷读 32/32 全量命中，昨日 §4.6 的缺口闭环；
- 2. 共享零性能代价：与 LocalDiskBackend 同口径对比，TTFT/吞吐/阵列供给（10.4 GB/s 峰值）全部持平，交叉读相对读自己仅 +5% TTFT；

3. 架构价值：预填充全机付一次、会话自由迁移、重启不丢缓存——这些能力建立在“共享介质 + 文件系统即索引”之上，本地盘方案在语义上无法提供；
4. 边界清晰：同机即插即用；跨节点需共享文件系统层或集中式索引，是下一步验证项。

附录 A：复现命令（同事服务器，容器 vllm 内）

A.1 双实例启动（共享池）

```
MODEL=/mnt/ws5000/models/Qwen3-Coder-480B-FP8
mkdir -p /mnt/ws5000/kvpool_fs
for I in 0 1; do
    DEVS=$( [ $I -eq 0 ] && echo "0,1,2,3" || echo "4,5,6,7"); PORT=$((8000+I))
    docker exec -d vllm bash -c "export HIP_VISIBLE_DEVICES=$DEVS VLLM_ROCM_USE_AITER=1 PYTHONHASHSEED=0 \
LMCACHE_CHUNK_SIZE=256 LMCACHE_LOCAL_CPU=True LMCACHE_MAX_LOCAL_CPU_SIZE=4 \
LMCACHE_REMOTE_URL=fs://local:0/mnt/ws5000/kvpool_fs LMCACHE_REMOTE_SERDE=naive; \
vllm serve $MODEL --served-model-name qwen \
--tensor-parallel-size 4 --enable-expert-parallel --trust-remote-code \
--max-model-len 32768 --gpu-memory-utilization 0.9 --no-enable-prefix-caching \
--kv-transfer-config '{"kv_connector":"LMCacheConnectorV1","kv_role":"kv_both"}' \
--port $PORT > /mnt/ws5000/fsws_i$I.log 2>&1"
    sleep 30
done
# 注意：URL 不能按官方文档写 fs:///path（该版本 parse_remote_url 强制校验 host:port 会失败），
# 必须写 fs://local:0/path 占位形式。
```

A.2 灌入与交叉冷读测量

```
# 并行灌入 48 会话（A: 0-23, B: 24-47）
docker exec -d vllm bash -c "python3 /mnt/ws5000/bench_mp.py 8000 populate 530 24 0 > /mnt/ws5000/result
s/fs_ppA.log 2>&1"
docker exec -d vllm bash -c "python3 /mnt/ws5000/bench_mp.py 8001 populate 530 24 24 > /mnt/ws5000/result
s/fs_ppB.log 2>&1"
# 交叉冷读（热共享证明）：A 读 B 灌的 24-39, B 读 A 灌的 0-15
sync; echo 3 | sudo tee /proc/sys/vm/drop_caches
iostat -x 1 400 /dev/md0 > /tmp/io_fs_CROSS.log &
docker exec -d vllm bash -c "python3 /mnt/ws5000/bench_mp.py 8000 measure 530 16 24 64 16 > /mnt/ws5000/re
sults/fs_CROSS_A.log 2>&1"
docker exec -d vllm bash -c "python3 /mnt/ws5000/bench_mp.py 8001 measure 530 16 0 64 16 > /mnt/ws5000/re
sults/fs_CROSS_B.log 2>&1"
# 取证（每实例 16 条请求应全部满额命中）：
grep -acE 'hit tokens: 30[0-9]{3}' /mnt/ws5000/fsws_i0.log
grep -acE 'hit tokens: 0,' /mnt/ws5000/fsws_i0.log # 期望 0
```

A.3 URL 校验 bug 的单测复现

```
docker exec vllm python3 -c "
from lmcache.v1.storage_backend.connector import parse_remote_url
parse_remote_url('fs:///mnt/ws5000/kvpool_fs') # AssertionError: missing host（文档示例写法）
parse_remote_url('fs://local:0/mnt/ws5000/kvpool_fs') # OK, path='/mnt/ws5000/kvpool_fs'
"
```

附录 B：原始数据存档（同事服务器）

文件	内容
/tmp/fsshare.out	实验全程编排日志
/mnt/ws5000/results/fs_{OWN_c8x2,OWN_c16x2,CROSS_c16x2}_{A,B}.log	各档两实例客户端原始输出
/mnt/ws5000/fsws_i{0,1}.log	两实例完整服务日志（含逐请求 hit 取证行、FS connector 初始化行）
/tmp/io_fs_*.log	各档 iostat 秒级原始记录
/mnt/ws5000/kvpool_fs/	共享池（344 GB / 22656 文件，实验后保留可复查）